

MegEngine1.4实现DTR的核心代码

代码地址：[imperative/src/impl/interpreter/interpreter_impl.cpp](#)

启发式公式

$$\frac{(\text{cost} + 10^{-3})^{\text{param_cost}} \cdot (\text{param_recompute_times})^{\text{recompute_times}}}{\left(\frac{\text{memory} + \text{free_mem}}{1024.0 \times 1024.0}\right)^{\text{param_mem}} \cdot (\text{cur_time} - \text{last_used_time} + 10^{-3})^{\text{param_time}}}$$

改进估价函数：我们对 DTR 的启发式估价函数做了一个小小的改进，引入了一些碎片相关的信息，希望换出的 tensor 除了自己占用的显存越大越好之外，还希望它在显存中两端的空闲显存块大小之和越大越好。

$$f(t) = \frac{\text{计算耗时}^{\alpha}}{\text{停留时长}^{\beta} (\text{显存} + \text{空闲段大小})^{\gamma}} \delta^{\text{重计算次数}}$$

此外我们还引入了**重计算次数**这一惩罚系数，希望每个算子被重算的次数尽量均匀。以及对于函数中的四个属性，增设了一些超参数，这样我们可以通过改变这些超参数来使启发式策略侧重于不同的属性。

重计算次数

```
1 void ChannelImpl::recompute(TensorInfo::ComputePath* path) {
2     SmallVector<TensorPtr> inputs;
3     inputs.reserve(path->inputs.size());
4     m_dtr.pin(path->inputs);
5     for (auto i : path->inputs) {
6         if (!i->ptr) {
7             regenerate(i); // 在 regenerate inputs 张量时, 可能会 OOM
8         }
9         inputs.push_back(i->ptr);
10        m_dtr.update_used_time(i); // 这里更新一下 update_used_time 合理吗?
11        // 如果一个张量在当前的计算路径中被用作输入, 那么它的使用时间应该被更新为当前时
        // 间, 这样在内存紧张时, 那些更久未使用的张量将更有可能被逐出。
12    }
13    if (m_worker_state.options.enable_dtr_auto_drop &&
        m_worker_state.options.dtr_eviction_threshold > 0) {
14        auto_evict();
15    }
16    auto outputs = OpDef::apply_on_physical_tensor(*path->op, inputs);
17    m_dtr.estimate_timestamp += path->compute_time / 1e8;
```

```

18     m_dtr.unpin(path->inputs);
19     for (size_t i = 0; i < outputs.size(); i++) {
20         auto&& o = path->outputs[i];
21         if (o) {
22             o->recompute_times++; // 重计算次数
23             if (!o->ptr) { // 如果输出张量的 ptr 为空（即之前被逐出了），则使用
produce_tensor 函数重新创建它。
24                 produce_tensor(o, std::move(outputs[i]), false);
25                 if (m_worker_state.options.enable_dtr_auto_drop) {
26                     m_dtr.update_dsu_after_recompute(o);
27                 }
28             }
29         }
30     }
31 }

```

空闲显存块大小

引入了一些碎片相关的信息，希望换出的 **tensor** 除了自己占用的显存越大越好之外，还希望它在显存中两端的空闲显存块大小之和越大越好。

获取显存碎片相关信息的过程是通过计算一个张量在其内存块两侧的空闲内存量来实现的。这种方法可以帮助确定逐出操作后可能获得的内存整理效果。

```

1  TensorInfo* ChannelImpl::DynamicSublinear::find_best_tensor() {
2      double min_msps = -1;
3      TensorInfo* best = nullptr;
4      for (auto i : candidates) {
5          if (i->producer && i->ptr && !i->pinned && i->evict_type ==
EvictType::NONE) {
6              double neighbor_cost = estimate_neighbor_cost(i);
7              // 获取 TensorInfo 对象所关联的张量数据的内存地址，并将这个地址以
size_t 类型的数值形式存储在变量 begin_ptr 中。
8              size_t begin_ptr = reinterpret_cast<size_t>(i->ptr->blob()-
>storage().get());
9              // 调用计算节点 (comp_node) 的 get_free_left_and_right 方法，传入张量
的起始地址和结束地址（起始地址加上大小）。
10             // 返回一个包含两部分空闲内存的 side_info:
11             // - side_info.first: 张量左侧的空闲内存量。
12             // - side_info.second: 张量右侧的空闲内存量。
13             auto side_info = i->ptr->
>comp_node().get_free_left_and_right(begin_ptr, begin_ptr + i->ptr->blob()-
>size());
14             // 将两侧的空闲内存量相加，得到张量两侧的总空闲内存量 free_mem。
15             double free_mem = side_info.first + side_info.second;

```

```

16         double msp = i->eval_func(neighbor_cost, free_mem,
    estimate_timestamp, 1.0, 1.0, 1.0, 1.0001);
17         if (min_msp < 0 || msp < min_msp) {
18             min_msp = msp;
19             best = i;
20         }
21     }
22 }
23 return best;
24 }

```

其中：

```

1  size_t begin_ptr = reinterpret_cast<size_t>(i->ptr->blob()->storage().get());

```

1. `i->ptr`：从 `TensorInfo` 指针 `i` 获取 `ptr` 成员，这是一个 `TensorPtr` 类型的智能指针，指向 `Tensor` 对象。
2. `ptr->blob()`：调用 `Tensor` 对象的 `blob()` 成员函数，返回对 `Blob` 对象的引用。`Blob` 管理着张量的底层存储。
3. `blob()->storage()`：调用 `Blob` 对象的 `storage()` 成员函数，返回对 `m_storage` 的引用。`m_storage` 是 `Blob` 类中的一个 `RawStorage` 类型成员，`RawStorage` 是 `std::shared_ptr<dt_byte>` 的别名，指向存储张量数据的字节数组。
4. `storage().get()`：`get()` 函数是 `std::shared_ptr` 提供的成员函数，它返回智能指针所管理的原始指针。这里，它返回指向 `dt_byte`（数据类型字节）数组的原始指针。
5. `reinterpret_cast<size_t>`：使用 `reinterpret_cast` 将原始指针转换为 `size_t` 类型的数值。这个转换得到的数值代表了指针的内存地址，以字节为单位。
6. 赋值给 `begin_ptr`：将转换得到的 `size_t` 类型的内存地址值赋给变量 `begin_ptr`。`begin_ptr` 现在存储了张量数据在内存中的起始地址。

✨ 根据：

```

using TensorPtr = std::shared_ptr<Tensor>;

TensorPtr ptr;

```

可知：

- `TensorPtr` 是一个智能指针，其底层实现是 `std::shared_ptr`，专门用于管理 `Tensor` 类型对象的生命周期。

- `ptr` 被声明为 `TensorPtr` 类型，意味着它是一个指向 `Tensor` 对象的智能指针。它是 `TensorInfo` 的成员变量，用于引用和控制该 `TensorInfo` 所代表的张量数据。其中，`TensorInfo` 用于跟踪张量的状态，并协助决定哪些张量可以被逐出以释放内存。

```
1  // m_storage 通过 DeviceTensorStorageTrait 所提供的分配和释放函数与具体的设备存储相
   关联。
2  class mgb::DeviceTensorStorageTrait {
3      public:
4          static void* alloc(CompNode node, size_t size) {
5              return node.alloc_device(size);
6          }
7
8          static void free(CompNode node, void *data) {
9              node.free_device(data);
10         }
11     };
12
13     template <class Trait>
14     class TensorStorage {
15     public:
16         using RawStorage = std::shared_ptr<dt_byte>;    // dt_byte 是一个代表数
   据类型字节的类型
17     };
18
19     class DeviceTensorStorageTrait;
20     // DeviceTensorStorage 是 TensorStorage 模板类的一个特化版本，使用
   DeviceTensorStorageTrait 作为其特性类。
21     using DeviceTensorStorage = TensorStorage<DeviceTensorStorageTrait>;
22
23     // Blob 类管理着张量的存储，而 Tensor 类则提供了一个高层的接口来访问和操作张量数据。
24     class Blob;
25     using BlobPtr = std::shared_ptr<Blob>;
26     class Blob : public NonCopyableObj {    // 表示张量的底层存储，与具体的设备（CPU、
   GPU等）相关联。
27     public:
28         using RawStorage = DeviceTensorStorage::RawStorage;
29         // 返回底层存储的引用。
30         const RawStorage& storage();
31     private:
32         // 这里的 m_storage 是一个 std::shared_ptr<dt_byte> 类型的智能指针，指向张量的
   底层字节数据。
33         mutable RawStorage m_storage;
34     };
35
```

```

36  const Blob::RawStorage& Blob::storage() {
37      if (!m_storage) {
38          BlobManager::inst()->alloc_with_defrag(this, m_size);
39      }
40      return m_storage;
41  }
42
43  class Tensor;
44  using TensorPtr = std::shared_ptr<Tensor>;
45  class Tensor : public NonCopyableObj { // 表示一个多维张量，可以是物理张量或虚拟张量。
46  public:
47      // 获取 Blob 的引用。
48      BlobPtr& blob() {
49          return m_blob;
50      }
51  private:
52      BlobPtr m_blob;
53  };

```

理解 `i->ptr->blob()->storage().get()` 的调用链：

1. **`i->ptr`** : `i` 是 `TensorInfo*` 类型的指针，`ptr` 是 `TensorInfo` 结构中的成员，它是 `TensorPtr` 类型，即 `Tensor` 类的智能指针。
2. **`ptr->blob()`** : `blob()` 是 `Tensor` 类的成员函数，返回对 `m_blob` 的引用，这里 `m_blob` 是 `BlobPtr` 类型，即 `Blob` 对象的智能指针。
3. **`blob()->storage()`** : `storage()` 是 `Blob` 类的成员函数，返回 `m_storage` 的引用，这里 `m_storage` 是 `RawStorage` 类型，即 `std::shared_ptr<dt_byte>`。
4. **`storage().get()`** : `get()` 是 `std::shared_ptr` 的成员函数，返回智能指针所管理的原始指针，这个原始指针指向张量在设备上的实际存储空间。

```

1  auto side_info = i->ptr->comp_node().get_free_left_and_right(begin_ptr,
    begin_ptr + i->ptr->blob()->size());

```

自上而下分析 `i->ptr->comp_node().get_free_left_and_right(begin_ptr, begin_ptr + i->ptr->blob()->size())` :

这行代码的作用是查询在特定内存区间两侧的空闲内存量。这个查询是通过 `CompNode` 和 `StreamMemAlloc` 接口委托给 `MemAllocImplHelper` 类实现的，该类维护了一个空闲内存块的映射，并提供了计算指定区间两侧空闲内存的方法。

1. Tensor 到 Blob 的转换：

- `i->ptr`：从 `TensorInfo` 结构中的 `ptr` 成员获取 `TensorPtr`，即 `std::shared_ptr<Tensor>`。
- `i->ptr->comp_node()`：调用 `Tensor` 类的 `comp_node()` 成员函数，该函数通过 `m_blob` 成员（`BlobPtr` 类型）访问 `Blob` 类的 `comp_node()`。

2. Blob 类的 `comp_node`：

- `Blob` 类的 `comp_node()` 成员函数返回与之关联的 `CompNode`。`CompNode` 是一个抽象类，表示计算资源。

3. `CompNode` 的 `get_free_left_and_right`：

- `comp_node().get_free_left_and_right(...)`：调用 `CompNode` 类的 `get_free_left_and_right` 成员函数。这个函数在编译配置 `!MGB_BUILD_SLIM_SERVING` 下才可用。

4. `CompNodeImpl` 的实现：

- `CompNode` 类中的 `get_free_left_and_right` 函数实际上是委托给 `ImplBase` 类的虚函数实现的。在 `CudaCompNode::CompNodeImpl` 类中，这个函数被重写，以调用与之关联的 `mem_alloc::StreamMemAlloc` 对象的 `get_free_left_and_right` 函数。

5. `StreamMemAlloc` 的 `get_free_left_and_right`：

- 在 `mem_alloc::StreamMemAlloc` 类中，`get_free_left_and_right` 函数是一个虚函数，它在 `MemAllocImplHelper` 类中有一个虚实现。在 `CudaCompNode::CompNodeImpl` 中，这个函数调用了 `m_mem_alloc->get_free_left_and_right(begin_ptr, end_ptr)`。

6. `MemAllocImplHelper` 的 `get_free_left_and_right`：

- `MemAllocImplHelper::get_free_left_and_right` 函数通过维护的内存块信息（`m_free_blk_addr` 和 `m_free_blk_size`）来计算指定内存区间 `[begin_ptr, end_ptr)` 两侧的空闲内存量。

7. 计算空闲内存：

- 函数首先锁定互斥体 `m_mutex` 以保证线程安全。
- 使用 `m_free_blk_addr` 来查找 `begin_ptr` 附近可能存在的空闲内存块。
- 计算 `begin_ptr` 左侧和右侧的空闲内存量，返回结果。

文件结构：

```

1  |—— imperative
2  |   |—— src
3  |   |   |—— include
4  |   |   |   |—— megbrain
5  |   |   |   |—— imperative
```

```

6 | | | | | | | — physical_tensor.h [1] ptr->comp_node() #include
  | | | | | | | "megbrain/tensor.h"
7 | — src
8 | | — core
9 | | | — impl
10 | | | | — comp_node
11 | | | | | — cuda
12 | | | | | | — comp_node.cpp [5] #include
  | | | | | | "megbrain/comp_node/alloc.h"
13 | | | | | | — comp_node.h
14 | | | | | | — mem_alloc
15 | | | | | | — alloc.cpp
16 | | | | | | — impl.cpp [7] #include "./impl.h"
17 | | | | | | — impl.h [6] #include "megbrain/comp_node/alloc.h"
18 | | | — include
19 | | | | — megbrain
20 | | | | | — comp_node
21 | | | | | | — alloc.h [4]
22 | | | | | | — comp_node.h [3] comp_node().get_free_left_and_right()
23 | | | | | | — tensor.h [2] #include "megbrain/comp_node.h"

```

imperative/src/include/megbrain/imperative/physical_tensor.h

```

1  #include "megbrain/tensor.h" // 定义了多维张量 (Tensor) 的类和相关操作
2
3  // Blob 类管理着张量的存储, 而 Tensor 类则提供了一个高层的接口来访问和操作张量数据。
4
5  class Blob;
6  using BlobPtr = std::shared_ptr<Blob>;
7  class Blob : public NonCopyableObj { // 表示张量的底层存储, 与具体的设备 (CPU、
  | GPU等) 相关联。
8  public:
9      // 返回计算节点的引用。
10     const CompNode& comp_node() const {
11         return m_comp_node;
12     }
13 private:
14     CompNode m_comp_node;
15 };
16
17 class Tensor;
18 using TensorPtr = std::shared_ptr<Tensor>;
19 class Tensor : public NonCopyableObj { // 表示一个多维张量, 可以是物理张量或虚拟张
  | 量。
20 public:

```

```

21     // 返回张量所在的计算节点。
22     CompNode comp_node() const {
23         mgb_assert(m_blob, "uninitialized tensor.");
24         return m_blob->comp_node();
25     }
26 private:
27     BlobPtr m_blob;
28 };

```

src/core/include/megbrain/tensor.h

```

1  #include "megbrain/comp_node.h"

```

src/core/include/megbrain/comp_node.h

```

1  /*!
2   * \brief abstraction of a streaming computing resource on localhost (a
3   * thread on CPU, a cuda stream, etc.)
4   *
5   * localhost 上 stream 计算资源的抽象 (CPU 上的线程、cuda 流等)
6   *
7   * Note that most of the operations are asynchronous with respect to the
8   * caller thread
9   * 请注意，大多数操作相对于调用者线程来说都是异步的
10  */
11 class CompNode {
12 public:
13     #if !MGB_BUILD_SLIM_SERVING
14         std::pair<size_t, size_t> get_free_left_and_right(size_t begin_ptr,
15 size_t end_ptr) {
16             return m_impl->get_free_left_and_right(begin_ptr, end_ptr);
17         }
18     #endif
19 protected:
20     //! ImplBase with env(); defined in CompNodeEnv
21     class Impl;
22     class ImplBase: public NonCopyableObj, public DynTypeObj {
23     public:
24
25     #if !MGB_BUILD_SLIM_SERVING

```



```

26         virtual std::pair<size_t, size_t>
get_free_left_and_right(size_t x, size_t y) {
27             return {x - x, y - y};
28         }
29
30 #endif
31
32     };
33
34     ///! implementations are allocated statically, so no memory management
is needed
35     // 实现是静态分配的, 因此不需要内存管理
36     ImplBase *m_impl = nullptr;
37
38 };

```

src/core/include/megbrain/comp_node/alloc.h

```

1  namespace mem_alloc {
2
3  /*!
4  * \brief interface for raw allocator 原始分配器的接口
5  *
6  * In case of allocation error, MemAllocError should be thrown 如果发生分配错
误, 应该抛出 MemAllocError
7  */
8  class RawAllocator {
9      public:
10         /*!
11         * \brief allocate memory of requested size (in bytes); if memory is
run out, return nullptr; in other cases of error, throw MemAllocError
12         * 分配请求大小的内存 (以字节为单位) ; 如果内存耗尽, 则返回 nullptr; 在其他错
误情况下, 抛出 MemAllocError
13         */
14         virtual void* alloc(size_t size) = 0;
15
16         /*!
17         * \brief release allocated memory 释放分配的内存
18         */
19         virtual void free(void *addr) = 0;
20
21         /*!
22         * \brief get free and total device memory 获取可用内存和总设备内存
23         * \param[out] free returned free memory in bytes 返回可用内存 (以字节为
单位)

```

```

24         * \param[out] tot returned total memory in bytes 返回总内存 (以字节为单
    位)
25         */
26         virtual void get_mem_info(size_t& free, size_t& tot) = 0;
27
28         virtual ~RawAllocator() = default;
29     };
30
31     /*!
32     * \brief like RawAllocator, but alloc() would not return nullptr; an
    exception is thrown if fails
33     * 与 RawAllocator 类似, 但 alloc() 不会返回 nullptr; 如果失败则抛出异常
34     */
35     class NonFallibleRawAllocator: public RawAllocator {
36     public:
37
38         /*!
39         * \brief allocate as shared_ptr that binds the deallocator 分配为绑定
    释放器的shared_ptr
40         */
41         virtual std::shared_ptr<void> alloc_shared(size_t size) {
42             auto del = [this](void* p) { free(p); };
43             return {alloc(size), del};
44         }
45     };
46
47     class MemAllocBase {
48     public:
49
50     #if !MGB_BUILD_SLIM_SERVING
51         /*!
52         * \brief get free memory adjacent to interval [begin_ptr, end_ptr]
53         * 获取与区间 [begin_ptr, end_ptr] 相邻的空闲内存
54         */
55         virtual std::pair<size_t, size_t> get_free_left_and_right(size_t
    begin_ptr, size_t end_ptr) {
56             return {0, 0};
57         }
58     #endif
59
60     };
61
62     class StreamMemAlloc: virtual public NonFallibleRawAllocator,
63                             virtual public MemAllocBase {
64     public:
65
66         /*!

```

```

67      * \brief allocate memory 分配内存
68      *
69      * Note that the caller is responsible to call cudaSetDevice before
        calling.
70      * 注意，调用者有责任在调用之前调用cudaSetDevice。
71      */
72      virtual void* alloc(size_t size) = 0;
73
74      virtual void free(void *addr) = 0;
75  };
76
77  } // mem_alloc

```

src/core/impl/comp_node/cuda/comp_node.cpp

```

1  #include "megbrain/comp_node/alloc.h"
2
3  /* ===== CudaCompNodeImpl ===== */
4  class CudaCompNode::CompNodeImpl final: public CompNode::Impl {
5
6      mem_alloc::StreamMemAlloc *m_mem_alloc;
7
8      #if !MGB_BUILD_SLIM_SERVING
9          std::pair<size_t, size_t> get_free_left_and_right(size_t begin_ptr,
10 size_t end_ptr) override {
11             return m_mem_alloc->get_free_left_and_right(begin_ptr, end_ptr);
12         }
13     #endif
14 };

```

src/core/impl/comp_node/mem_alloc/impl.h

`MemAllocImplHelper` 类是一个辅助类，用于实现内存分配器的基础功能。通过维护空闲内存块的有序集合，它支持快速的内存查找、分配和释放操作。

内存管理逻辑：

- `MemAllocImplHelper` 类通过维护两个映射（`m_free_blk_size` 和 `m_free_blk_addr`）来跟踪内存中的空闲块。
- 当内存被**释放**时，它尝试找到相邻的空闲块，并将它们合并为一个更大的空闲块。
- 当内存被**分配**时，它根据需要拆分空闲块或分配一个现有的空闲块。
- 通过 `m_free_blk_size` 映射，可以快速找到足够大的空闲块来满足分配请求。

- 通过 `m_free_blk_addr` 映射，可以快速定位特定的内存地址，并获取相关的空闲块信息。

`m_free_blk_size` 和 `m_free_blk_addr` 是 `MemAllocImplHelper` 类中的两个关键数据结构，它们共同维护着内存分配器中的空闲内存块信息。这两个映射之间的关系和作用如下：

1. `m_free_blk_size` 映射：

- 这个 `std::map` 按照空闲内存块的大小排序（使用 `FreeCmpBySize` 作为比较函数），键是 `FreeBlock` 结构，它包含内存块的起始地址和大小。
- 值是 `BlkByAddrIter` 结构，它存储了一个迭代器，指向 `m_free_blk_addr` 映射中对应的条目。

2. `m_free_blk_addr` 映射：

- 这个 `std::map` 按照内存块的起始地址排序，键是内存块的起始地址，值是 `FreeBlockAddrInfo` 结构。
- `FreeBlockAddrInfo` 结构中包含了内存块的 `is_head` 标志、大小 `size`，以及一个指向 `m_free_blk_size` 映射的迭代器 `siter`。

3. 关系：

- `m_free_blk_size` 和 `m_free_blk_addr` 通过迭代器和内存块的地址相互关联。`m_free_blk_size` 中的每个条目都指向 `m_free_blk_addr` 中相应条目的地址。
- `m_free_blk_addr` 中的 `siter` 成员变量存储了一个迭代器，指向 `m_free_blk_size` 中对应的条目，这样就形成了双向链接。

4. 作用：

- `m_free_blk_size` 允许分配器快速找到足够大的空闲内存块来满足分配请求，因为它是按照内存块大小排序的。
- `m_free_blk_addr` 允许分配器根据内存地址快速定位空闲内存块，并获取相关信息，如是否是头部块、内存块的大小等。

5. 使用场景：

- 当需要分配内存时，分配器首先在 `m_free_blk_size` 中查找足够大的空闲块。
- 一旦找到合适的空闲块，分配器通过 `BlkByAddrIter` 中的迭代器在 `m_free_blk_addr` 中找到对应的地址条目。
- 在内存释放或合并空闲块时，分配器会更新这两个映射来保持空闲内存信息的一致性。

6. 同步更新：

- 当内存块被分配或释放时，可能需要在两个映射中插入、删除或更新条目。这些操作通常需要同时修改 `m_free_blk_size` 和 `m_free_blk_addr` 来保持数据的一致性。

```
1 #include "megbrain/comp_node/alloc.h"
```

```

2
3 class MemAllocImplHelper: virtual public MemAllocBase {
4
5     protected:
6         struct MemAddr { // 表示内存地址
7             ///! whether it is head of a chunk from raw allocator; if true, it
could not be merged with chunks with lower address
8             /// 是否是来自原始分配器的 chunk 头; 如果为 true, 则无法与较低地址的
chunks 合并
9             bool is_head = false;
10            size_t addr = -1;
11
12            void* addr_ptr() const { // 将 addr 转换为 void* 类型的指针
13                return reinterpret_cast<void*>(addr);
14            }
15
16            bool operator < (const MemAddr &rhs) const { // 重载小于运算符,
用于比较两个 MemAddr 对象
17                return addr < rhs.addr;
18            }
19
20            MemAddr operator + (size_t delta) const { // 重载加法运算符, 用于地
址加上一个偏移量
21                return {false, addr + delta};
22            }
23        };
24
25        struct FreeBlock { // 表示一个空闲内存块
26            MemAddr addr; // 块的起始地址
27            size_t size = -1; // 块的大小
28
29            size_t end() const { // 返回块的结束地址
30                return addr.addr + size;
31            }
32        };
33
34        struct FreeCmpBySize{ // 比较两个 FreeBlock 对象的大小, 如果大小相同则比
较地址, 确保可以得到一个按大小排序的块集合。
35            bool operator() (const FreeBlock &a, const FreeBlock &b) const {
36                /// prefer more recent (hotter) block 更喜欢最近 (更热) 的区块
37                return a.size < b.size || (a.size == b.size && a.addr <
38                    b.addr);
39            }
40        };
41
42        ///! free blocks sorted by size, and map to corresponding iterator in
m_free_blk_addr

```

```

42         // 空闲块按大小排序，并映射到 m_free_blk_addr 中相应的迭代器
43         // 键是 FreeBlock，值是 BlkByAddrIter。它按大小排序空闲块，并且使用
        FreeCmpBySize 作为比较函数。
44         std::map<FreeBlock, BlkByAddrIter, FreeCmpBySize> m_free_blk_size;
45
46         //! map from address to size and size iter 从地址映射到 size 和 size
        iter
47         // 键是空闲块的起始地址，值是 FreeBlockAddrInfo。它映射地址到块大小和对应的迭
        代器。
48         std::map<size_t, FreeBlockAddrInfo> m_free_blk_addr;
49
50         std::mutex m_mutex; // 使用互斥锁来保证线程安全。
51
52         struct BlkByAddrIter {
53             decltype(m_free_blk_addr.begin()) aiter; // 指向
        m_free_blk_addr 映射的迭代器。
54         };
55
56         struct FreeBlockAddrInfo {
57             bool is_head; //! always equals to siter->first.addr.is_head 始
        终等于 siter->first.addr.is_head
58             size_t size;
59             decltype(m_free_blk_size.begin()) siter; // 指向
        m_free_blk_size 映射的迭代器
60         };
61
62         #if !MGB_BUILD_SLIM_SERVING
63             std::pair<size_t, size_t> get_free_left_and_right(size_t begin_ptr,
        size_t end_ptr) override;
64         #endif
65
66     };

```

src/core/impl/comp_node/mem_alloc/impl.cpp

```

1     #include "./impl.h"
2
3     #if !MGB_BUILD_SLIM_SERVING
4     std::pair<size_t, size_t> MemAllocImplHelper::get_free_left_and_right(size_t
        begin_ptr, size_t end_ptr) {
5         // m_mutex 确保在查找空闲内存块时不会有其他线程修改数据结构。
6         MGB_LOCK_GUARD(m_mutex);
7
8         // 在 m_free_blk_addr 映射中查找 ≥ begin_ptr 的迭代器 iter，利用了
        m_free_blk_addr 映射的有序性。

```

```

9      // 该迭代器指向刚好大于 begin_ptr 的紧邻的下一个元素
10     auto iter = m_free_blk_addr.lower_bound(begin_ptr);
11     // 初始化左右两侧的空闲内存量为 0
12     size_t left_free = 0, right_free = 0;
13
14     // 计算左侧空闲内存
15     if (iter != m_free_blk_addr.begin()) { // 如果当前迭代器 iter 不是
m_free_blk_addr 的开始，则检查前一个块。
16         auto prev = iter;
17         prev --;
18         // 如果前一个块的结束地址 (prev->first + prev->second.size) 正好是
begin_ptr，则说明左侧有一块空闲内存，其大小为 prev->second.size。
19         if (prev->first + prev->second.size == begin_ptr) {
20             left_free = prev->second.size;
21         }
22     }
23     // 计算右侧空闲内存
24     if (iter != m_free_blk_addr.end()) { // 如果当前迭代器 iter 不是
m_free_blk_addr 的结束，则检查当前块。
25         // 如果当前块的起始地址正好是 end_ptr，则说明右侧有一块空闲内存，其大小为
iter->second.size。
26         if (iter->first == end_ptr) {
27             right_free = iter->second.size;
28         }
29     }
30     return {left_free, right_free};
31 }
32 #endif

```

`MemAllocImplHelper::get_free_left_and_right` 函数的目的是找出给定内存区间 `[begin_ptr, end_ptr)` 两侧的空闲内存量。这个函数是 `MemAllocImplHelper` 类中定义的，用于内存分配器的辅助实现。